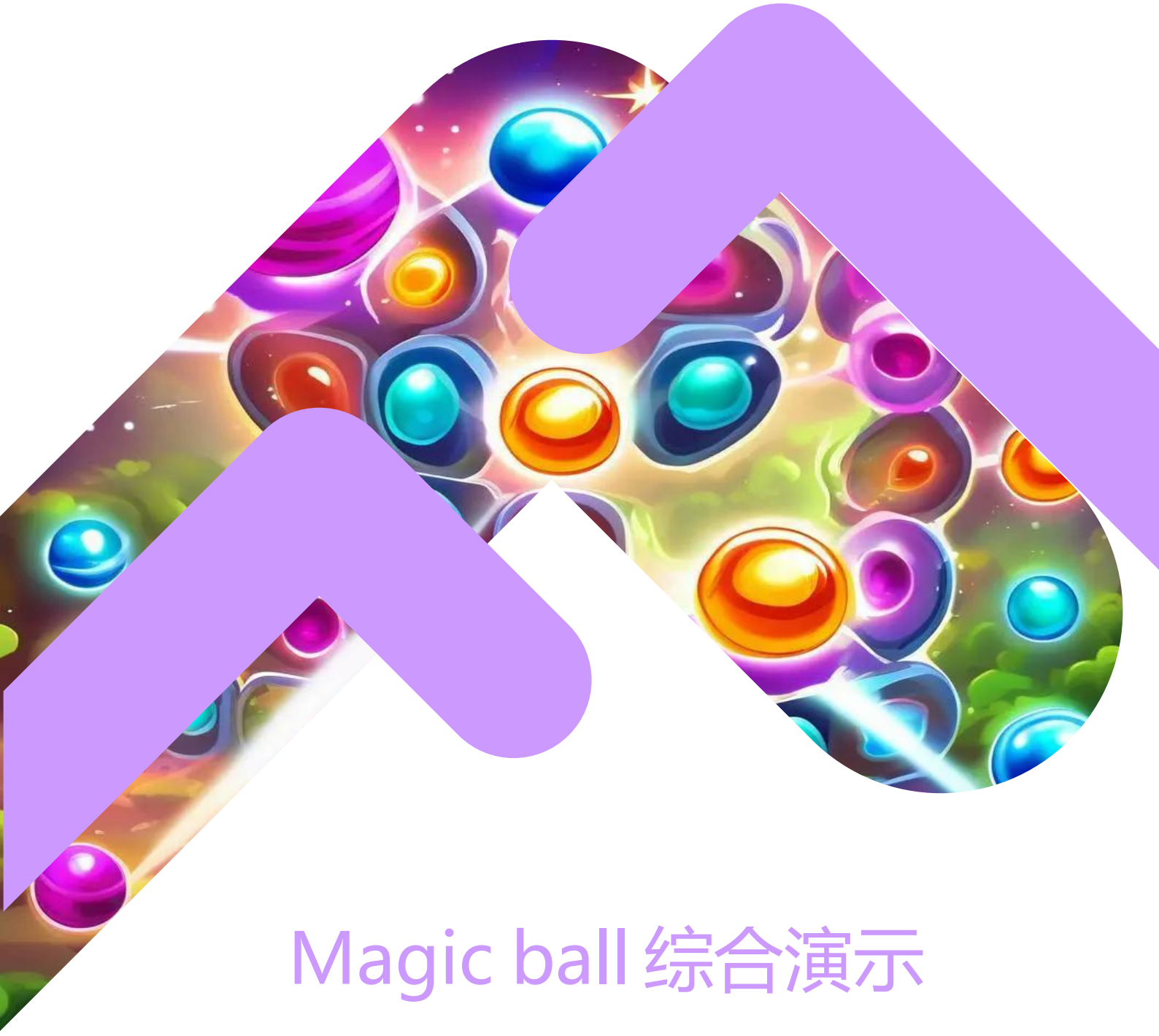


Magic ball 综合演示实验报告

信 12　2350222 盛曦 2024.6.11

1. 题目（黑体，小三号，段前段后间距各1行）

1.1.次级小标题（如果有，宋体，四号，加粗，段前段后间距各1行）

1.1.1. 三级小标题（如果有，宋体，小四，加粗，段前段后间距各1行）

制作一个magic ball彩球连连看游戏，需要实现：
根据输入的行列（5-9）生成一个二维的彩球数组
在数组中求出并标出彩球中可以消去的部分（数字与伪图形部分）
初始的消去、下落、除零的内部数组实现及伪图形动画展示
新值填充的内部数组实现及伪图形动画展示
查找可消除提示项，并标出
游戏功能，包括鼠标的操作反馈、积分等等
游戏过程中的消去、下落、除零等内部数组及伪图形实现

2. 整体设计思路

用一个二维数组ball记录随机生成的彩球，用一个二维数组ball1记录各个位置的彩球的属性，0表示无属性，1表示可消去，2表示消除提示。ball1、ball、x、y组成一个结构体，方便函数调用。

整体思路可以分为以下步骤：

一：菜单的打印，输入处理： menu函数返回选择了哪个模式

二：彩球数组的行列xy输入及输入错误处理：input函数

三：生成彩球数组：ball内元素随机生成

四：伪图形的绘制：若是初始图形，则需先调整窗口大小及字体，再进行边框及彩球绘制，分为有分隔线和无分割线两种；若不是初始图形，则只需画彩球（因为后续的操作都只会改变彩球）

五：寻找并标记可消去的项并消去：寻找可消除项后将ball1中的相应位置置零。TF_eliminate函数用于寻找并记录可消去的项，并绘制伪图形的消除动画。

六：下落除零：fall函数实现。先下落：将ball1中属性为1的彩球，在ball中依次变为0（ball1中也要置零），同时伪图形动画展示出消去的过程；再除零：把ball中大小为0的彩球随机生成，并用伪图形动画展示

七：寻找消除提示项：eliminate_edge和eliminate_middle两个函数寻找消除提示项，并将ball1中对应的位置设置为2，表示消除提示项，并在为图形界面用同心圆表示。

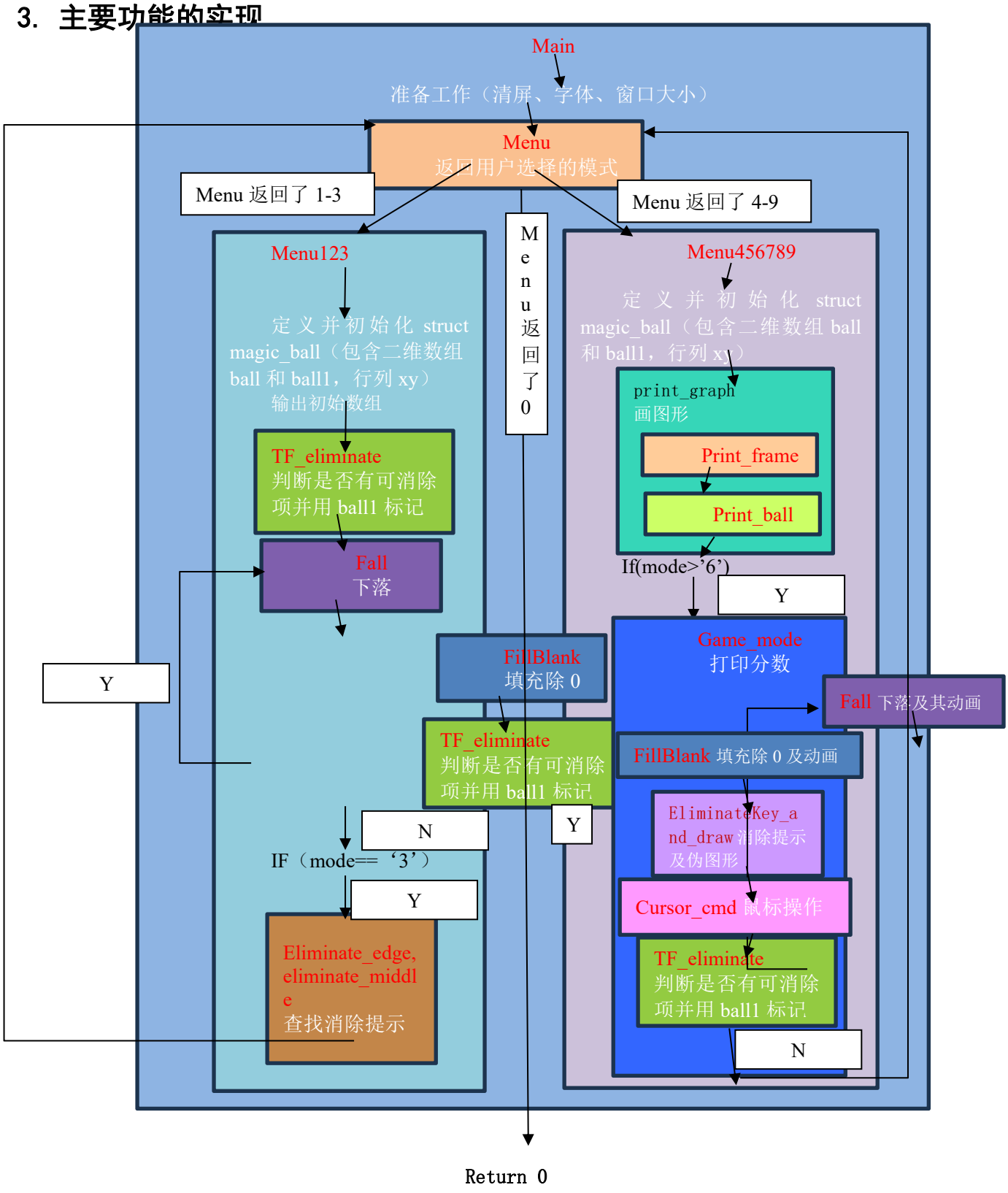
八：鼠标指令。用户的鼠标行为分为这几种：左键：没有选中球/选了不可以选的球/选了可以选的球但不能消去/选了可以选的球且可以消去/取消选择，右键：退出。根据鼠标指令重复四、五、六、七项。用corsor_cmd函数实现。

九：页面的改变及提示词的输出：包括各种提示词、分数等

十：判断游戏是否结束：用TF_eliminateKey函数判断ball中有无消除提示项（即ball1中有没有2）；

鼠标是否右键

3. 主要功能的实现



上图的说明：
彩色方框表示函数
红字表示函数名称
白字表示步骤
黑字表示判断

部分重要函数内部说明：

1，TF_eliminate

通过循环，找出ball数组中可以消去的位置（3个及以上连在一起的，实际只需要判断3个连在一起就可以），并在ball1中用数字1标记，返回值为int型，1表示有可消去的，0表示无可消去的

2，Fall

下落，先找出ball1中标记为1的位置，依次将ball中相应位置的数字置零，即消去彩球，并做出伪图形动画展示：彩球闪烁一段时间后消失变为空白；然后将ball中的0“上浮”，方法是：以ball中的一竖列为例，flag先置为0，表示还没有碰到0，从ball中的这一列下往上依次判断，若碰到了0，则把flag置1，flag为1了以后，再在ball中碰到非0（例如5）且该非0元素下方的元素为0，则需“下落”这个5，即将5与其下一个0不断交换，直至5的下方不是0，表示5已经下落至底部，重复执行这个下落5的操作，直到0以上再没有非0

以下给出函数部分代码作为示例：

```
int flag = 0;//flag为1，表示当前位置的下方有0，即当前位置需要下落
for (i = magic.y - 1; i >= 0; i--)
{
 if (flag == 1 && magic.ball[i][j] != 0 && magic.ball[i + 1][j] == 0) //非0的下方是0
 {
     for (k = i; k + 1 < magic.y && magic.ball[k + 1][j] == 0; k++)//不断下落直到非0下降到底部
     {
         exchange(&magic.ball[k][j], &magic.ball[k + 1][j]);
         exchange(&magic.ball1[k][j], &magic.ball1[k + 1][j]);
         …伪图形的下落动画，略
     }
 }
}
```

3，FillBlank

通过循环找到数组ball中为0的位置（即消除后的位置），依次生成随机值填充彩球，并做出伪图形的动画展示

4，eliminate_middle

寻找消除提示项。本函数负责寻找中间的情况：一个位置的周围四格中有三格在数组范围内且相同，则将这一格和那三格中的中间那一格交换即可，因此将数组ball1中这两个可交换的位置置2

5， eliminate_edge

寻找消除提示项。本函数负责寻找边缘的情况：一个位置的上/下/左/右中有一个方向上有连着2（或以上）个格子相同，且与这个位置的相反方向上的1（或多）格也相同，则将相反方向的这一格与该位置交换即可，因此将数组ball1中这两个可交换的位置置2

6， EliminateKey_and_draw

综合eliminate_middle和eliminate_edge，然后print_ball（为了打印出消除提示项的不同）

7， game_mode

函数主体为一个while(1)循环，负责记录分数的变化，显示下落除零及其动画，然后判断是否有消除提示项：若无则游戏结束，直接跳出menu456789函数，回到main函数；若有则进入cursor_cmd函数，让用户进行鼠标操作。

8， cursor_cmd

本函数用于返回用户的鼠标指令，在game_mode中被调用。

在一个while(1)循环中不断读取鼠标的位置和鼠标操作，初始状态为“未选择”

若用户左键单击且位置合法，则选中了一个球，分为以下几种：

与上一次选择的一样：取消选择，状态变为“未选择”，回到循环开头重新读取用户的选择

与上一次选择不一样且不相邻：无法消除，选择位置改变，回到循环开头重新读取用户的选择

与上一次选择不一样，相邻，但无法消除：输出提示词“不能交换”，选择位置改变，回到循环开头重新读取用户的选择

与上一次选择不一样，相邻，且可消除：状态变为“未选择”（其实没有必要），跳出本函数，回到game_mode函数中while(1)循环的开始，进行下落除零动画，然后判断有无消除提示项，若有则继续鼠标指令的循环

4. 调试过程碰到的问题

4.1.很难找出消除提示项的错误

因为没有一个demo来判断消除提示项是否是错的，而如果用自己写的程序判断可能会陷入自己先前所犯过的错误，而且不好区分一个被标记了消除提示的位置是“只与周围部分消除提示项交换可消除”还是“与任何位置交换都不可消除”（即错误的消除提示项）。因此基本上只能用肉眼判断。

解决方法：因为最终的要求是消除提示项要用双层的圆来画，但是双层圆看不清楚，所以可以用更清楚的图形来显示消除提示项，如方块等，这样在肉眼判断的时候更清楚；也可以让身边的人

玩自己写的游戏，帮忙检查有没有出错的地方，正如大部分的产品都有的用户反馈渠道一样，使用得越多越能发现问题。

4.2.有一些函数调用的参数不必要/重复

一些函数，特别是伪图形部分使用的函数，经常调用差不多的参数，比如调用了数组ball，通常就要调用数组ball1，数组的行和列xy，直接调用这些参数需要重复地写相同的内容，会很麻烦。
解决方法：定义结构体，包含了ball、ball1、x、y，这样每次调用时就不需要在形参列表写很多不必要的东西了。但是个人觉得结构体或许可以换一个方式，由两个int型变量组成，一个表示彩球编号，一个表示彩球状态（普通，可消除，可消除提示），整个彩球系统由一个9*9的结构体数组组成，这样在交换彩球的时候不用把两个数组ball和ball1中的元素都交换，只要把两个结构体交换一下就可以了，可惜程序已经完成了大半，而且实际上交换彩球的地方并不多，因此没有再改。

4.3.函数的拆分比较容易出错

在一开始写程序时，是将菜单8和9的功能放在一个函数中实现的，但是这样会导致这一个函数过于臃肿，且一旦其中一些板块出了问题（比如cursor_cmd），则很难定位出问题的地方在哪里。在写完菜单8和9的所有功能后，再进行函数的拆分，好不容易将基本每个函数的行数都降到了50行，再调试时却发现程序出问题了。
解决方法：在动笔写程序前，先大致的梳理一遍程序的整个流程，大概需要依次实现哪些功能，这些功能该怎样分配到各个函数去，可以列草稿，想清楚以后再动手写代码，这样就不会一股脑全写在一个函数里。此外，在对代码中已经成功实现功能的部分进行改动时，最好先对没有问题的代码进行备份，这样万一在改了很久以后发现又出问题了，可以和原先没问题的代码进行对照，快速找到问题。

4.4.鼠标右键后有弹窗

在菜单8和9中，鼠标右键后会出现弹窗，导致每次在输入“end”结束以前必须先左键单击一下exe窗口。即使enable了鼠标的功能也没有用。
解决方法：先是跟老师的test_cct进行对照，因为老师的test中右键是不会有弹窗的，但对照后并没有找到关键性的区别所在；然后仔细检查了右键前后的代码，尤其是右键之后的，因为右键后会立马要你输入end，因此怀疑可能是输入方式的问题，于是使用了get开头的三种方法、cin及其成员函数的一些列方法、fgets等，都没有办法解决问题。最后在不断尝试下，发现在右键后Sleep一段时间（不能太短），先disable鼠标前再enable一下鼠标，就可以解决问题了。这次经验告诉我们要不断尝试，且要与正确的代码不断比对。

5. 心得体会

5.1. 预先构思

在写程序之前，应该先大致地在脑海中过一遍程序的运行过程，从而判断出应该大致分为哪几个函数，并依据这个写一写草稿，这样有助于在之后正式写程序的时候进行合理的功能划分，而不会想到什么些什么，想到什么功能就一味地加进去，导致程序很乱，函数分配不合理，改也要时间。合理地划分函数后，再实现后续功能，添加内容时，会显得更加清晰。

5.2. 多做注释

不但要在变量、函数的命名等方面做到简明易懂，也应该多多写注释。比如在一个很长的while循环结束后，在}后注释一个end of while(1)，可以帮助我们阅读程序，在该while循环内部碰到continue、break语句等时，可以快速找到接下来会跳转到哪里。

多做注释不止让自己看得懂自己写的程序，也可以让其他人阅读懂。万一要和别人进行合作，清晰的注释、简单易懂的变量和函数命名、宏定义等，都可以让合作者更快地领会意思。

6. 附件：源程序

```
int TF_eliminate(struct magic_ball& magic)//判断并记录可消除项
{
    int i, j, m;
    int flag = 0;//记录是否有可消除项
    init_all(magic.ball1, 0);
    for (i = 0; i < magic.y; i++){
        for (j = 0; j < magic.x; j++){
            for (m = -2; m <= 0; m++){
                if (j + m >= 0 && j + m < magic.x - 2 && magic.ball[i][j + m] == magic.ball[i][j + m + 1]
&& magic.ball[i][j + m] == magic.ball[i][j + m + 2]){
                    magic.ball1[i][j] = 1;
                    flag = 1;
                }
                if (i + m >= 0 && i + m < magic.y - 2 && magic.ball[i + m][j] == magic.ball[i + m + 1][j]
&& magic.ball[i + m][j] == magic.ball[i + m + 2][j]){
                    magic.ball1[i][j] = 1;
                    flag = 1;
                }
            }
        }
    }
    return flag;
}
void fall(struct magic_ball& magic,char mode)//下落除0
```

```
{
    int i, j, k;
    int graph = (mode == '0' ? 0 : 1);//graph缺省为0，不绘制伪图形
    for (j = 0; j < magic.x; j++){
        for (i = 0; i < magic.y; i++){
            if (magic.ball1[i][j] == 1){
                magic.ball[i][j] = 0;
            }
        }
        int flag = 0;//flag为1，表示当前位置的下方有0，即当前位置需要下落
        for (i = magic.y - 1; i >= 0; i--){
            if (flag == 1 && magic.ball[i][j] != 0 && magic.ball[i + 1][j] == 0)//当前不为0，当前位置的下方一个为0，这时候才需要下落；因为刚进循环的时候flag为0，不可能进这个if，所以横坐标i+1不会越界
            {
                for (k = i; k + 1 < magic.y && magic.ball[k + 1][j] == 0; k++){
                    exchange(&magic.ball[k][j], &magic.ball[k + 1][j]);
                    exchange(&magic.ball1[k][j], &magic.ball1[k + 1][j]);
                    if (graph != 0){
                        int line_parm =2;
                        cct_showch(2 + j * 2 * line_parm, 2 + k * line_parm, ' ', COLOR_HWHITE, COLOR_HWHITE, 2);
                        cct_showstr(2 + j * 2 * line_parm, 2 + (k + 1) * line_parm-1, "○", magic.ball[k + 1][j], COLOR_BLACK, 1);
                        Sleep(20);//一个球掉下半个格子，停顿
                        cct_showstr(2 + j * 2 * line_parm, 2 + k * line_parm+1, "═", COLOR_HWHITE, COLOR_BLACK,1 );
                        cct_showstr(2 + j * 2 * line_parm, 2 + (k+1) * line_parm, "○", magic.ball[k+1][j], COLOR_BLACK, 1);
                        Sleep(20);//一个球掉下一个格子，停顿
                    }
                }
                Sleep(40);//一个球的掉落过程完毕，停顿
            }
            if (magic.ball[i][j] == 0){
                flag = 1;
            }
        }
    }
}
void eliminate_middle(struct magic_ball& magic)
{
    int i, j;
    for (i = 0; i < magic.y; i++){
        for (j = 0; j < magic.x; j++){
            if (i - 1 >= 0 && j - 1 >= 0 && i + 1 < magic.y && magic.ball[i - 1][j] == magic.ball[i + 1][j] && magic.ball[i - 1][j] == magic.ball[i][j - 1])
            {
                magic.ball1[i][j] = 2;
                magic.ball1[i][j - 1] = 2;
            }
            …下略
        }
    }
}

void eliminate_edge(struct magic_ball& magic)
{
```

```
    int i, j;
    for (i = 0; i < magic.y; i++){
        for (j = 0; j < magic.x; j++){
            if (i + 2 < magic.y && magic.ball[i + 1][j] == magic.ball[i + 2][j]){
                if (i - 1 >= 0 && magic.ball[i - 1][j] == magic.ball[i + 1][j]){
                    magic.ball1[i][j] = 2;
                    magic.ball1[i - 1][j] = 2;
                }
                …下略
            }
        }
}
void eliminate_graph(struct magic_ball& magic, char mode, int& score, int TF_add_score)
{
    int line_parm = (mode == '4' || mode == '6' ? 1 : 2);
    int k;
    struct location loc;
    cct_setcursor(CURSOR_INVISIBLE);
    for (loc.i = 0; loc.i < magic.y; loc.i++){
        for (loc.j = 0; loc.j < magic.x; loc.j++){
            if (magic.ball1[loc.i][loc.j] == 1){
                if (TF_add_score)
                    score++;
                for (k = 0; k < 7; k++){
                    print_one_ball("○", magic.ball, loc, line_parm);
                    Sleep(30);
                    print_one_ball("□", magic.ball, loc, line_parm);
                    Sleep(30);
                }
                cct_showch(2 + loc.j * 2 * line_parm, 2 + loc.i * line_parm, ' ', COLOR_HWHITE, COLOR_HWHITE,
2);
                Sleep(50);
            }

        }
    }
    cct_setcursor(CURSOR_VISIBLE_NORMAL);
}
int pre_oneball(struct location& preloc, struct location& loc, struct magic_ball& magic, int line_parm, char
mode)//上一次有选中一个ball
{
    if (loc.i == preloc.i && loc.j == preloc.j)//和上次选中的一样
    {
        print_one_ball("◎", magic.ball, loc, line_parm);
        preloc.i = -1;
        preloc.j = -1;
    }
    else//和上次选中不一样
    {
        if (fabs(preloc.i - loc.i) == 1 && fabs(preloc.j - loc.j) == 0 || fabs(preloc.i - loc.i) == 0 &&
fabs(preloc.j - loc.j) == 1)//相邻
        {
            exchange(&magic.ball[loc.i][loc.j], &magic.ball[preloc.i][preloc.j]);
            if (TF_eliminate(magic) == 0){
                exchange(&magic.ball[loc.i][loc.j], &magic.ball[preloc.i][preloc.j]);
                EliminateKey_and_draw(magic, mode);//每次交换ball中元素后，都应该调用一下，保证ball1中的
元素不要错位
```

```
                print_one_ball("◎", magic.ball, loc, line_parm, COLOR_HWHITE);
                print_one_ball("◎", magic.ball, preloc, line_parm);
                print_bottom(magic.y, line_parm, "不能交换");
                cout << char('A' + loc.i) << "行" << 1 + loc.j << "列" << "<=>" << char('A' + preloc.i) <<
"行" << 1 + preloc.j << "列";
                Sleep(250);
                preloc.i = loc.i;
                preloc.j = loc.j;
            }
            else//可以消除
            {
                print_current_select(magic.y, line_parm, loc);
                preloc.i = -1;
                preloc.j = -1;
                return 1;
            }
        }
        else//选中的和上次不相邻
        {
            print_one_ball("◎", magic.ball, loc, line_parm, COLOR_HWHITE);
            print_one_ball("◎", magic.ball, preloc, line_parm);
            print_current_select(magic.y, line_parm, loc);
            preloc.i = loc.i;
            preloc.j = loc.j;
        }
    }
    return 0;
}
void cursor_cmd(struct magic_ball& magic, int line_parm, char mode, struct location& preloc, int& end)
{
    cct_enable_mouse();
    int X = 0, Y = 0;
    int maction, keycode1, keycode2;
    while (1) {
        if (cct_read_keyboard_and_mouse(X, Y, maction, keycode1, keycode2) == CCT_MOUSE_EVENT) {
            int flag_pos = TF_pos_legal(X, Y, magic, line_parm);//记录位置是否合法,1表示合法
            struct location loc;
            loc.i = Y / 2 - 1;
            loc.j = X / 4;
            if (flag_pos == 1) {//合法位置
                if (maction == MOUSE_LEFT_BUTTON_CLICK) {//左键
                    if (magic.ball1[loc.i][loc.j] != 2){
                        print_bottom(magic.y, line_parm, "不能选择");
                        print_bottom(magic.y, line_parm, "", 11);
                        cout << char('A' + loc.i) << "行" << 1 + loc.j << "列";
                        Sleep(250);
                        continue;
                    }
                    else {
                        if (mode == '9') {
                            if (preloc.i != -1){//上一次有选择一个ball
                                if (pre_oneball(preloc, loc, magic, line_parm, mode))
                                    break;
                            }
                            else {//上次没有选中一个ball
                                print_one_ball("◎", magic.ball, loc, line_parm, COLOR_HWHITE);
                                print_current_select(magic.y, line_parm, loc);
```

```
                                preloc.i = loc.i;
                                preloc.j = loc.j;
                            }
                        }
                        if (mode == '8') {
                            print_current_select(magic.y, line_parm, loc);
                            Sleep(250);
                            break;
                        }
                    }
                }
                else if (maction == MOUSE_RIGHT_BUTTON_CLICK) {//右键
                    Sleep(100);//不加这个右键就会有弹窗
                    end = 1;
                    break;
                }//其余键（双击等）不管
            }
        }//键盘操作不管
    }//end of while(1)
    cct_enable_mouse();//不加这个右键就会有弹窗
    cct_disable_mouse();
}
void eliminate_and_fill(struct magic_ball& magic, char mode, int& score, int TF_add_score, int line_parm)
{
    while (TF_eliminate(magic) != 0)//下落除0，新值填充
    {
        print_ball(magic, mode);
        if (mode == '7')
            wait_for_enter("按回车键进行消除及下落除0操作...", magic.y, line_parm, 1);
        eliminate_graph(magic, mode, score, TF_add_score);
        fall(magic, mode);
        if (mode == '7')
            wait_for_enter("按回车键进行新值填充...", magic.y, line_parm, 1);
        FillBlank(magic, mode);
    }
}
void game_mode(struct magic_ball& magic, char mode, int line_parm)
{
    int score = 0;
    int TF_add_score = 0;//用于记录当前是否为初始消除，若为初始消除则不加score
    while (1){
        int end = 0;//用于记录右键退出
        eliminate_and_fill(magic, mode, score, TF_add_score, line_parm);
        if (mode >= '8'){
            cct_showstr(15, 0, "(当前分数:", COLOR_BLACK, COLOR_WHITE, 1);
            cout << score << " 右键退出)";
            TF_add_score = 1;
        }
        struct location preloc;
        preloc.i = -1;
        preloc.j = -1;//用于记录上一次选中的ball，初始为-1表示没有选中，因此放在下落除0后初始化
        if (mode == '7')
            wait_for_enter("按回车键显示消除提示...", magic.y, line_parm, 1);
        EliminateKey_and_draw(magic, mode);//消除提示，画新球
        if (mode == '9' && TF_eliminateKey(magic.ball1) == 0){
            cct_showstr(15, 0, "(无可消除项，游戏结束！)", COLOR_BLACK, COLOR_WHITE, 1);
            break;
```

```
        }
        if (mode >= '8'){
            cct_setcursor(CURSOR_INVISIBLE);
            cursor_cmd(magic, line_parm, mode, preloc, end);
            if (mode == '9' && TF_eliminate(magic) == 1)
                continue;
            cct_setcursor(CURSOR_VISIBLE_NORMAL);
        }//end of if(mode>='8')
        if (mode != '9' || mode == '9' && end == 1)
            break;
    }//end of while(1)
}
```